



Spring Framework I

-by Miss. Geetanjali Gajbhar

Spring Core

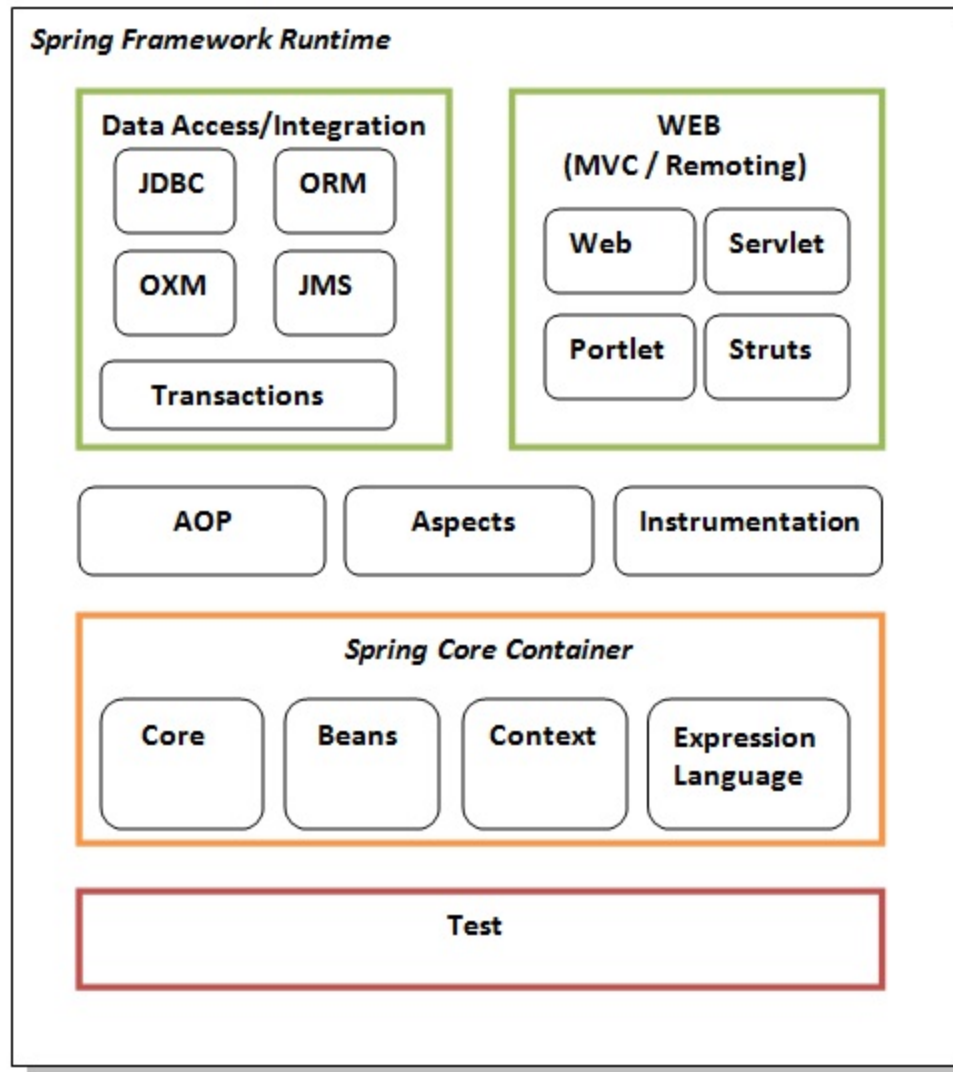
Spring Framework Modules

1. **Spring core** → Dependency Injection, Inversion of control.
2. **Spring Data Integration** → Spring JDBC, Spring ORM.
3. **Spring Web** → To create a Web application.
4. **Spring MVC** → To write the code in a layered Structure.
5. **Test** → Provide support to perform unit testing.

What is Spring?

1. **Framework Of Frameworks** → Because we can easily integrate with multiple frameworks (Ex. Hibernate, JDBC)

2. It is a **modularized framework**
3. **Dependency Injection Framework** → To make Java application loosely coupled.
4. **Lightweight Framework** → supports POJO classes.
5. It is a **non-invasive** Framework.
6. Spring provides **IOC (Inversion of control) container** which is responsible to automatically inject the dependency.
7. We can easily develop **J2EE Applications**.
8. It was developed by Rod Johnson in 2003.



Chapter 1

Introduction

Dependency Injection(D.I.)

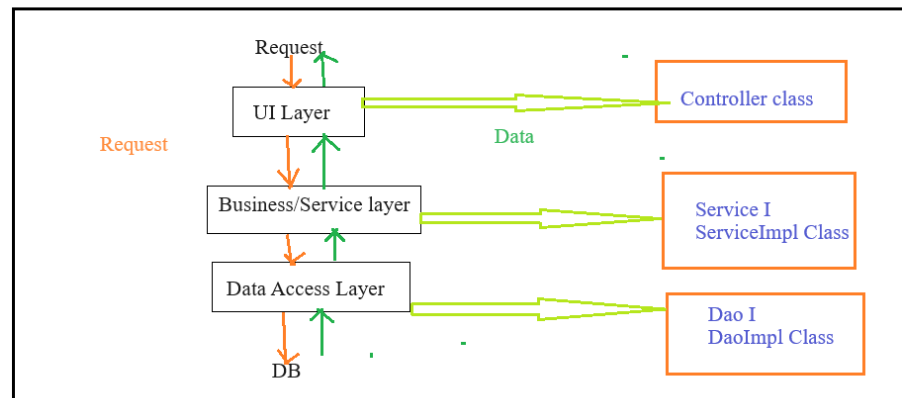
- We have seen Secondary references in the class. We injected another class manually and we created the objects using a new keyword manually. But in the Spring framework, We have an IOC container that will automatically inject the required dependencies, and create objects as well.
- So we can say in **Spring Framework we have an IOC container that will perform DI internally.**
- Dependency injection is a **design pattern** in our program, data type/member are nothing but dependency. Dependency injection makes our program code **loosely coupled**. And easier for testing. We provide information from the external source such as **xml file**

Inversion of Control (IOC)

- Spring provides an IOC container that will automatically **create objects and inject the dependencies as per requirements at runtime.**
- For this, we need to provide **metadata** to an IOC Container using either an XML approach or an Annotation approach.

Application Layers

Explain how to inject dependencies in different layers.



Chapter 2

Spring IOC Container

- It is a predefined Program.

Use:

- To create the object, and hold the object in memory.
- DI as per requirements
- Maintain the life cycle of Objects/beans. (From object creation to the destruction of the object)

What metadata do we have to provide to the IOC container?

- beans → POJO classes
- Configuration → Bean dependency

Types of IOC Container

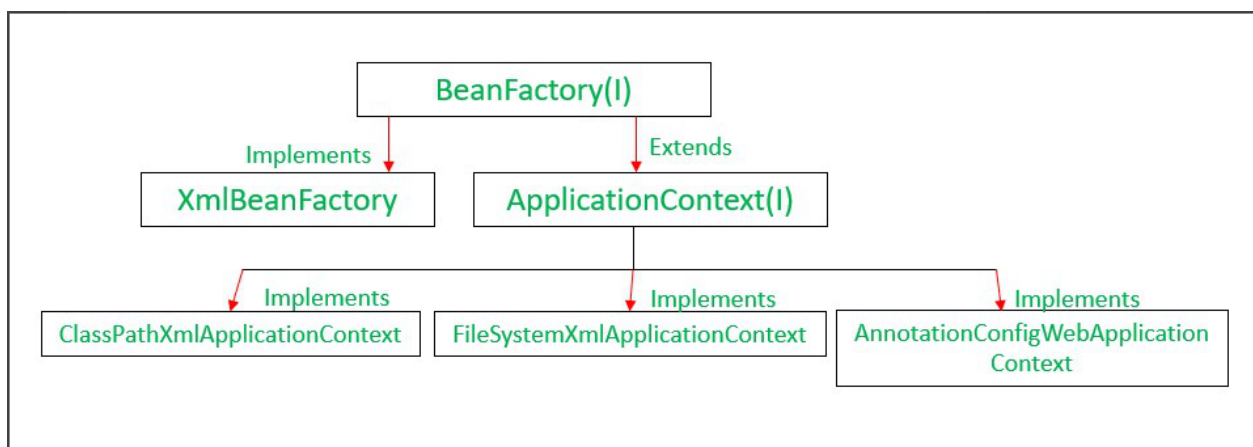
1. BeanFactory(I)
2. ApplicationContext(I)

BeanFactory (I) or Core Container

- Implemented class is **XmlBeanFactory**.
- *BeanFactory* is deprecated from Spring 3.0.

ApplicationContext(I) or J2EE Container

- Implemented classes are :
1. **ClassPathXmlApplicationContext**
 2. **FileSystemXmlApplicationContext**
 3. **AnnotationConfigWebApplicationContext**



Difference between Bean Factory and Application Context

Bean Factory	Application Context
It is a fundamental container For managing the beans	It is an advanced container that extends the Bean Factory and adds some advanced features.
suitable to build standalone applications.	Web applications
It does not support Annotations.	It supports Annotation based configuration in Bean Auto Wiring.
Bean Factory by default provides lazy loading	Application Context by default provides eager loading

Doesn't Support Internationalization	Support Internationalization
---	------------------------------

Required Jar files for Spring Core:

1. Spring AOP
2. Spring Aspect
3. Spring Beans
4. Spring Context
5. Spring Context indexer
6. Spring Context support
7. Spring Core
8. Spring Expression
9. Spring Instrument
10. Spring Jcl

Examples:

Note: We will use XML-based Configuration here.

- Step1: Create a New java Project
- Step 2: Add all required jar files for Spring IOC in the build path.
- Step 3: Create model class Student in package com.model
- Step 4:- Create xml file on SRC level. →bean.xml
- Step 5:- Create Test class use Core container(Bean Factory) in com.client package

Using Bean Factory

- Package com.model

Student

```
public class Student {  
  
    public Student() {  
        System.out.println("Constructor get called!!");  
    }  
  
}
```

- bean.xml on SRC level

```
<?xml version="1.0" encoding="UTF-8"?>  
<beans xmlns="http://www.springframework.org/schema/beans"  
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
    xsi:schemaLocation="  
        http://www.springframework.org/schema/beans  
        http://www.springframework.org/schema/beans/spring-beans.xsd">  
  
    <bean id="stu" class="com.model.Student">  
  
    </bean>  
</beans>
```

- Package com.client

Test Class

```
import org.springframework.beans.factory.BeanFactory;  
import org.springframework.beans.factory.xml.XmlBeanFactory;  
import org.springframework.core.io.ClassPathResource;  
import org.springframework.core.io.Resource;
```

```

import com.model.Student;

public class Test {
    public static void main(String[] args) {

        //load the resource manually
        Resource r=new ClassPathResource("bean.xml");

        //Beanfactory OC
        BeanFactory bf=new XmlBeanFactory(r);

        Student st = (Student)bf.getBean("stu");
        System.out.println("Constructor get called it means
        beans get created automatically");

    }

}

```

OutPut:

Constructor get called!!

Constructor get called it means beans get created automatically

Using Application Context

- Everything is same just a small change in the Test class

Test

```

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicat:
import org.springframework.core.io.ClassPathResource;
import org.springframework.core.io.Resource;

import com.model.Student;

```



```

public class Test {
    public static void main(String[] args) {
        ApplicationContext apc=
        new ClassPathXmlApplicationContext("bean.xml");

        Student st = (Student)apc.getBean("stu");
        System.out.println("Constructor get called it
        means beans get created automatically");

    }
}

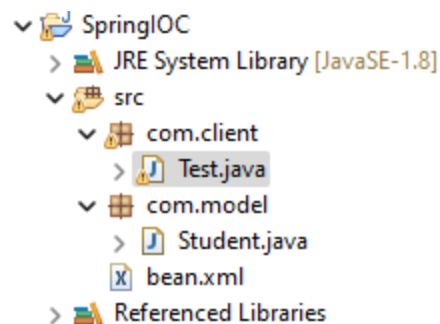
```

Output:

Constructor get called!!

Constructor get called it means beans get created automatically.

Your project structure should look like this



Chapter 3

Dependency Injection

Ways to inject the dependency:

1. **Setter-based injection/ Property injection**
2. **Constructor based injection**

Datatypes/Dependencies:

1. **Primitive Type**
2. **Collection Type**
3. **Reference Type/Non-primitive type/Secondary reference.**

Setter based Injection:

Primitive Type DI

Student class

```
package com.model;

public class Student {
    private int id;
    private String name;
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
}
```

```
}
```

```
}
```

Bean.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="
    http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/
    spring-beans.xsd">

  <bean id="stu" class="com.model.Student">
    <property name="id" value="101"></property>
    <property name="name" value="Aboli"></property>

  </bean>

</beans>
```

Test class

```
package com.client;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicat:
import org.springframework.core.io.ClassPathResource;
import org.springframework.core.io.Resource;
```

```

import com.model.Student;

public class Test {
    public static void main(String[] args) {
        ApplicationContext apc=
            new ClassPathXmlApplicationContext("bean.xml");

        Student st = (Student)apc.getBean("stu");
        System.out.println(st.getId());
        System.out.println(st.getName());

    }
}

```

101

Aboli

Secondary type DI

Student class

```

package com.model;

public class Student {
    private int id;
    private String name;
    private Address addr;

    //generate getter setter
    public int getId() {
        return id;
    }
    public void setId(int id) {

```

```

        this.id = id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public Address getAddr() {
        return addr;
    }
    public void setAddr(Address addr) {
        this.addr = addr;
    }
}

```

Address Class

```

package com.model;

public class Address {
    private int addId;
    private String addName;
    public int getAddId() {
        return addId;
    }
    public void setAddId(int addId) {
        this.addId = addId;
    }
    public String getAddName() {
        return addName;
    }
    public void setAddName(String addName) {
        this.addName = addName;
    }
}

```

```
}  
  
}
```

bean.xml

```
<?xml version="1.0" encoding="UTF-8"?>  
<beans xmlns="http://www.springframework.org/  
schema/beans"  
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
  xsi:schemaLocation="  
    http://www.springframework.org/schema/beans  
    http://www.springframework.org/schema/beans/  
    spring-beans.xsd">  
  
  <bean id="stu" class="com.model.Student">  
    <property name="id" value="101"></property>  
    <property name="name" value="Aboli"></property>  
    <property name="addr" ref="ad"></property>  
  </bean>  
  
  <bean id="ad" class="com.model.Address">  
    <property name="addId" value="50001"></property>  
    <property name="addName" value="surat"></property>  
  </bean>  
  
</beans>
```

Test class

```
package com.client;  
  
  
  
import org.springframework.context.ApplicationContext;  
import org.springframework.context.support.ClassPathXmlApplicat:
```

```

import org.springframework.core.io.ClassPathResource;
import org.springframework.core.io.Resource;

import com.model.Student;

public class Test {
    public static void main(String[] args) {
        ApplicationContext apc=
        new ClassPathXmlApplicationContext("bean.xml");

        Student st = (Student)apc.getBean("stu");
        System.out.println(st.getId());
        System.out.println(st.getName());
        System.out.println(st.getAddr().getAddId());
        System.out.println(st.getAddr().getAddName());

    }
}

```

Collection Type DI

Student class

```

package com.model;

import java.util.List;
import java.util.Map;
import java.util.Set;

public class Student {
    private List<String> list;
    private Set<Integer> set;
    private Map<Integer,String> map;
}

```

```

//generate getter Setter
public List<String> getList() {
    return list;
}
public void setList(List<String> list) {
    this.list = list;
}
public Set<Integer> getSet() {
    return set;
}
public void setSet(Set<Integer> set) {
    this.set = set;
}
public Map<Integer, String> getMap() {
    return map;
}
public void setMap(Map<Integer, String> map) {
    this.map = map;
}
}

```

bean.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/
schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="
http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/
spring-beans.xsd">

```



```

<bean id="stu" class="com.model.Student">

    <!-- List Values -->
    <property name="list">
    <list>
    <value>Aboli</value>
    <value>Akshada</value>
    <value>Pratiksha</value>
    </list>
    </property>

    <!-- Set Values -->
    <property name="set">
    <set>
    <value>555</value>
    <value>444</value>
    <value>333</value>
    </set>
    </property>

    <!-- Map Values -->

    <property name="map">
    <map>
    <entry key="111" value="Omkar"></entry>
    <entry key="222" value="Sameer"></entry>
    <entry key="333" value="Sam"></entry>
    </map>
    </property>

</bean>

<bean id="ad" class="com.model.Address">
<property name="addId" value="50001"></property>
<property name="addName" value="surat"></property>
</bean>

```

```
</beans>
```

Test Class

```
package com.client;

import java.util.List;
import java.util.Map;
import java.util.Set;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import org.springframework.core.io.ClassPathResource;
import org.springframework.core.io.Resource;

import com.model.Student;

public class Test {
    public static void main(String[] args) {
        ApplicationContext apc=
            new ClassPathXmlApplicationContext("bean.xml");

        Student st = (Student)apc.getBean("stu");
        List<String> list = st.getList();

        for(String s:list) {
            System.out.println(s);
        }
        Set<Integer> set = st.getSet();

        for(Integer i:set) {
            System.out.println(i);
        }
    }
}
```

```

        Map<Integer, String> map = st.getMap();
        Set<Integer> keys = map.keySet();
        for(Integer key:keys) {
            String value = map.get(key);
            System.out.println(value);
        }

    }

}

```

Aboli
 Akshada
 Pratiksha
 555
 444
 333
 Omkar
 Sameer
 Sam

Constructor Based Injection

Primitive DI

Student class

```

package com.model;

import java.util.List;
import java.util.Map;
import java.util.Set;

public class Student {
    private int id;

```

```

private String name;

//generate constructor using fields
public Student(int id, String name) {
    super();
    this.id = id;
    this.name = name;
}

//override toString method
@Override
public String toString() {
    return "Student [id="+id+",name="+name +"]";
}

```

bean.xml

```

<bean id="stu" class="com.model.Student">
<constructor-arg value="101"></constructor-arg>
<constructor-arg value="Aboli"></constructor-arg>
</bean>

```

Test Class

```

package com.client;

import java.util.List;
import java.util.Map;
import java.util.Set;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicat:
import org.springframework.core.io.ClassPathResource;
import org.springframework.core.io.Resource;

```

```

import com.model.Student;

public class Test {
    public static void main(String[] args) {

        ApplicationContext apc =
        new ClassPathXmlApplicationContext("bean.xml");
        Student st = (Student) apc.getBean("stu");
        System.out.println(st);

    }

}

```

Student [id=101, name=Aboli]

Secondary Reference DI

Student class

```

package com.model;

import java.util.List;
import java.util.Map;
import java.util.Set;

public class Student {
    private int id;
    private String name;
    private Address addr;

    //constructor
    public Student(int id, String name, Address addr) {
        super();
        this.id = id;
    }
}

```

```

        this.name = name;
        this.addr = addr;
    }
    //toString

    @Override
    public String toString() {
        return "Student [id="+id+",name="+name+",addr="+addr+"]";
    }

```

Address class

```

package com.model;

public class Address {
    private int addId;
    private String addName;

    //constructor
    public Address(int addId, String addName) {
        super();
        this.addId = addId;
        this.addName = addName;
    }

    //toString

    @Override
    public String toString() {
        return "Address[addId="+addId+",addName="+addName+"]";
    }

}

```

bean.xml

```
<bean id="ad" class="com.model.Address">
<constructor-arg value="50001"></constructor-arg>
<constructor-arg value="Pune"></constructor-arg>
</bean>

<bean id="stu" class="com.model.Student">
<constructor-arg value="101"></constructor-arg>
<constructor-arg value="Aboli"></constructor-arg>
<constructor-arg ref="ad"></constructor-arg>
</bean>
```

Test.class

```
package com.client;

import java.util.List;
import java.util.Map;
import java.util.Set;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplication:
import org.springframework.core.io.ClassPathResource;
import org.springframework.core.io.Resource;

import com.model.Student;

public class Test {
    public static void main(String[] args) {

        ApplicationContext apc =
        new ClassPathXmlApplicationContext("bean.xml");
        Student st = (Student) apc.getBean("stu");
        System.out.println(st);
    }
}
```

```
}  
  
}
```

Student [id=101, name=Aboli, addr=Address [addId=50001, addName=Pune]]

Difference between SBI and CBI

Setter Based Injection(SBI)	Constructor based Injection(CBI)
Partial Injection possible	Not possible
SBI overrides CBI	can't

Chapter 4

Spring Bean Scope

- Scope of bean refers to **lifecycle of beans**
- Spring provides 5 bean scope

For desktop based application

1. **Singleton** → Single object in a project and same object will be shared for each request made for that bean. ex.SessionFactory
2. **Prototype** → For every request one new object will be created.

For web based application

1. **Request**
2. **Session**
3. **Global Session**

[Note- By default spring framework use **singleton design pattern**]

Proof

Student Class

```
public class Student {  
    private int id;  
    private String name;  
  
    //getter setter  
  
}
```

bean.xml

```
<bean id="stu" class="com.model.Student">  
  <property name="id" value="101"></property>  
  <property name="name" value="aboli"></property>  
</bean>
```

Test Class

```
package com.client;  
  
import java.util.List;  
import java.util.Map;  
import java.util.Set;  
  
import org.springframework.context.ApplicationContext;  
import org.springframework.context.support.ClassPathXmlApplicationContext;  
import org.springframework.core.io.ClassPathResource;  
import org.springframework.core.io.Resource;  
  
import com.model.Student;  
  
public class Test {
```

```

public static void main(String[] args) {

    ApplicationContext apc =
    new ClassPathXmlApplicationContext("bean.xml");

    Student st1 = (Student) apc.getBean("stu");
    Student st2 = (Student) apc.getBean("stu");

    System.out.println(st1.hashCode());
    System.out.println(st2.hashCode());

}

}

```

1234776885

1234776885

Hence proved it is following Singleton scope by default

How to change the scope?

Use same Studetn class And Test Class

bean.xml

```

<bean id="stu" class="com.model.Student"
    scope="prototype">
    <property name="id" value="101"></property>
    <property name="name" value="aboli"></property>
</bean>

```

Test class

```

public class Test {
    public static void main(String[] args) {

```

```

        ApplicationContext apc =
new ClassPathXmlApplicationContext("bean.xml");
        Student st1 = (Student) apc.getBean("stu");
        Student st2 = (Student) apc.getBean("stu");

        System.out.println(st1.hashCode());
        System.out.println(st2.hashCode());

    }

}

```

1776957250

1268066861

Hence proved it is following prototype scope

Lazy loading and eager loading

- **Lazy Loading:** Load the object when getter method are called or when required.

Eager

Loading: Load all the objects when code executes.

Scenario I When [Scope=Singleton]

1. Bean Factory acts as Lazy Loading
2. **Application Context acts as Eager Loading**

Scenario II When [Scope=Prototype]

1. Bean Factory acts as Lazy Loading
2. Application Context acts as Lazy Loading

Chapter 5

Autowiring

- Spring container can inject secondary dependency by using **Autowiring**.
- By default Autowiring is **disabled**.
- Autowiring can be achieved by following ways:
 1. **byName** → **bean name**
 2. **byType** → **Class type**
 3. **Constructor**
 4. **autoDetect (below 3.5)**
 5. **none (by default)**

By name:

- This will inject the dependency according to **name of the bean**.
- **Note:** Bean id and Reference name **must be same**

Student class

```
package com.model;

import java.util.List;
import java.util.Map;
import java.util.Set;

public class Student {
    private int id;
    private String name;
    private Address addr;/**This reference name

    //getter setter
    public int getId() {
```

```

        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public Address getAddr() {
        return addr;
    }
    public void setAddr(Address addr) {
        this.addr = addr;
    }
}

```

Address Class

```

package com.model;

public class Address {
    private int addId;
    private String addName;

    //getter setter
    public int getAddId() {
        return addId;
    }
    public void setAddId(int addId) {
        this.addId = addId;
    }
    public String getAddName() {

```

```

        return addName;
    }
    public void setAddName(String addName) {
        this.addName = addName;
    }
}

```

bean.xml

```

<bean id="addr" class="com.model.Address">
<property name="addId" value="50001"></property>
<property name="addName" value="pune"></property>
</bean>

<bean id="stu" class="com.model.Student"
autowire="byName">
<property name="id" value="101"></property>
<property name="name" value="Aboli"></property>

</bean>

```

Test.class

```

package com.client;

import java.util.List;
import java.util.Map;
import java.util.Set;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicat:
import org.springframework.core.io.ClassPathResource;
import org.springframework.core.io.Resource;

```

```

import com.model.Student;

public class Test {
    public static void main(String[] args) {

        ApplicationContext apc =
        new ClassPathXmlApplicationContext("bean.xml");
        Student st = (Student) apc.getBean("stu");
        System.out.println(st.getId());
        System.out.println(st.getName());
        System.out.println(st.getAddr().getAddId());
        System.out.println(st.getAddr().getAddName());

    }

}

```

101
Aboli
50001
pune

byType

- Here dependency is injected as per the **type of bean**. (Class of bean)
- Here we **don't need** to give reference name and bean id same.
- But there must be only one object of that type. Otherwise **ambiguity** will be there.
- Example:

Refer the Student and Address class, Test class from above

bean.xml

```

<bean id="ad" class="com.model.Address">
<property name="addId" value="50001"></property>
<property name="addName" value="pune"></property>
</bean>

<bean id="stu" class="com.model.Student"
    autowire="byType">
<property name="id" value="101"></property>
<property name="name" value="Aboli"></property>

</bean>

```

Note:

If multiple bean with same class are found, ambiguity will occur. To resolve this add tag

"autowire-candidate="false", now that bean will not get involved in autowiring process.

Example:

- Use same Student ,Address and Test class as above

bean.xml

```

<bean id="ad1" class="com.model.Address"
    autowire-candidate="false">

<property name="addId" value="50001"></property>
<property name="addName" value="pune"></property>
</bean>

<bean id="ad2" class="com.model.Address">
<property name="addId" value="60001"></property>

```



```

<property name="addName" value="Mumbai"></property>
</bean>

<bean id="stu" class="com.model.Student"
autowire="byType">
<property name="id" value="101"></property>
<property name="name" value="Aboli"></property>

</bean>

```

Now Only the bean with id **ad2** will get created.

Constructor

- The constructor mode injects the dependency by calling the constructor of the class.
- It internally uses 'byType', but while injecting property it uses **constructor based injection**.

student class

```

package com.model;

import java.util.List;
import java.util.Map;
import java.util.Set;

public class Student {
    private int id;
    private String name;
    private Address addr;

    //constructor
    public Student(int id, String name, Address addr) {
        super();
    }
}

```

```

        this.id = id;
        this.name = name;
        this.addr = addr;
    }

    //toString
    @Override
    public String toString() {
    return "Student[id="+id+",name="+name+",addr="+addr+"]";
    }

}

```

Address Class

```

package com.model;

public class Address {
    private int addId;
    private String addName;
    //constructor
    public Address(int addId, String addName) {
        super();
        this.addId = addId;
        this.addName = addName;
    }
    //toString
    @Override
    public String toString() {
    return "Address[addId="+addId+",addName="+addName+"]";
    }

}

```

bean.xml

```

<bean id="ad" class="com.model.Address">
<constructor-arg name="addId" value="50001">
</constructor-arg>
<constructor-arg name="addName" value="pune">
</constructor-arg>
</bean>

<bean id="stu" class="com.model.Student"
    autowire="constructor">
<constructor-arg name="id" value="101">
</constructor-arg>
<constructor-arg name="name" value="Aboli">
</constructor-arg>

</bean>

```

Student [id=101, name=Aboli, addr=Address [addId=50001, addName=pune]]

Autowiring with Annotation

- **@Autowired** annotation for spring bean autowiring.
- Autowired internally uses **byType**, hence to avoid ambiguity we use **@Qualifier** annotation with **@Autowired**
- To enable this annotation we have to do some configuration.

Student class

```

package com.model;

import java.util.List;
import java.util.Map;
import java.util.Set;

```

```

import org.springframework.beans.factory.annotation.Autowired;

public class Student {
    private int id;
    private String name;

    @Autowired                      //*****
    private Address addr;

    //getter setter
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public Address getAddr() {
        return addr;
    }
    public void setAddr(Address addr) {
        this.addr = addr;
    }
}

```

Address Class

```
package com.model;

public class Address {
    private int addId;
    private String addName;

    //getter setter
    public int getAddId() {
        return addId;
    }
    public void setAddId(int addId) {
        this.addId = addId;
    }
    public String getAddName() {
        return addName;
    }
    public void setAddName(String addName) {
        this.addName = addName;
    }
}
```

bean.xml

```
<!-- Configure the annotation class-->

<bean class="org.springframework.beans.factory.
annotation.AutowiredAnnotationBeanPostProcessor">
</bean>

<bean id="ad" class="com.model.Address">
<property name="addId" value="50001"></property>
<property name="addName" value="pune"></property>
```

```

</bean>

<bean id="stu" class="com.model.Student">
<property name="id" value="101"></property>
<property name="name" value="Aboli"></property>

</bean>

```

Test class

```

package com.client;

import java.util.List;
import java.util.Map;
import java.util.Set;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import org.springframework.core.io.ClassPathResource;
import org.springframework.core.io.Resource;

import com.model.Student;

public class Test {
    public static void main(String[] args) {

        ApplicationContext apc =
            new ClassPathXmlApplicationContext("bean.xml");
        Student st = (Student) apc.getBean("stu");
        System.out.println(st.getId());
        System.out.println(st.getName());
        System.out.println(st.getAddr().getAddId());
        System.out.println(st.getAddr().getAddName());
    }
}

```

```
}
```

101
Aboli
50001
pune

What is @Qualifier annotation?

Ans: @Autowired Annotation internally used **byType**, Hence if multiples beans of same type are defined in XML file then it will show **ambiguity error**, hence to avoid that we use @Qualifier annotation with @Autowired.

bean.xml

```
<!--If we have defined 2 Objects of-->

<bean id="ad1" class="com.model.Address">
  <property name="addId" value="60001"></property>
  <property name="addName" value="Mumbai"></property>
</bean>

<bean id="ad2" class="com.model.Address">
  <property name="addId" value="60001"></property>
  <property name="addName" value="Mumbai"></property>
</bean>
```

Student class

```
public class Student {
    private int id;
    private String name;

    @Autowired
    @Qualifier("ad1")
```

```
private Address addr;  
  
}
```

Chapter 6

Singleton Class

What is Singleton Class?

A singleton class means we can create **only one object** of that class.

It is a **design pattern**.

How to create Singleton class?

- Using **Factory methods**
- Factory Methods : Methods that returns instance of **its own class**.

A class

```
package Singleton_Class;  
  
public class A {  
  
    //private variable  
    private static final A a=new A();  
  
    //private constructor  
    private A()  
    {  
        System.out.println("private constructor");  
    }  
}
```



```

    }

    //Factory Method
    public static A getA(){
        System.out.println("factory method ");
        return a;
    }
    public void msg(){
        System.out.println("hello user");
    }
}
}

```

bean.xml

```

<bean id="a" class="Singleton_Class.A"
factory-method="getA"></bean>

```

Test Class

```

package Singleton_Class;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicat:

public class Test {
    public static void main(String[] args) {
        ApplicationContext context=
        new ClassPathXmlApplicationContext("bean.xml");
        A a=(A)context.getBean("a");
        a.msg();
    }

}

```

private constructor
factory method
hello user

Chapter 7

Stereotype Annotations

- We can replace the XML configuration by the Annotations

Topic	XML	Annotations
Bean Creation	<Bean >	@Component or @Bean
DI	<property>/ <construct-args>	@Value
Scope	<Bean scope="prototype">	@Scope
Scan the base package	<context:component-scan >	@ComponentScan
For configuration		@Configuration
to inject secondary class		@Autowired

Example 1:

Student Class

```
package com.model;

import java.util.List;
import java.util.Map;
import java.util.Set;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.beans.factory.annotation.Value;
```

```

import org.springframework.stereotype.Component;

@Component("stu")
public class Student {
    @Value("101")
    private int id;
    @Value("Aboli")
    private String name;

    // -----
    // generate getter setter
    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}

```

bean.xml

```

<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:context="http://www.springframework.org/schema/context"

```

```

xmlns:p="http://www.springframework.org/schema/p"
xsi:schemaLocation="
    http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans.xsd
    http://www.springframework.org/schema/context
    http://www.springframework.org/schema/context/spring-context.xsd"

<context:component-scan base-package="com.model"/>

</beans>

```

Test Class

```

package com.client;

import java.util.List;
import java.util.Map;
import java.util.Set;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import org.springframework.core.io.ClassPathResource;
import org.springframework.core.io.Resource;

import com.model.Student;

public class Test {
    public static void main(String[] args) {

        ApplicationContext apc =
            new ClassPathXmlApplicationContext("bean.xml");
        Student st1 = (Student) apc.getBean("stu");
        System.out.println(st1.getId());
    }
}

```

```

        System.out.println(st1.getName());

    }

}

```

101
Aboli

Example 2: Without XML file

student class

```

@Component("stu")
public class Student {

    public Student() {
        System.out.println("Student class Constructor");
    }
}

```

JavaConfig class

```

@Configuration
@ComponentScan("com.model")
public class JavaConfig {

}

```

Test Class

```

public class Test {
    public static void main(String[] args) {

        ApplicationContext apc =

```

```

new AnnotationConfigApplicationContext(JavaConfig.class);
    Student st1 = (Student) apc.getBean("stu");

}

}

```

Student class Constructor

Example 3: Without using @Component

Student Class

```

public class Student {

    public Student() {
        System.out.println("Student class Constructor");
    }
}

```

JavaConfig Class

```

@Configuration
public class JavaConfig {

    @Bean("stu")
    public Student getStudent() {
        Student s=new Student();
        return s;
    }
}

```

Test class

```
public class Test {  
    public static void main(String[] args) {  
  
        ApplicationContext apc =  
new AnnotationConfigApplicationContext(JavaConfig.class);  
        Student st1 = (Student) apc.getBean("stu");  
  
    }  
  
}
```

Student class Constructor

- **If we use @Bean then no need to write @ComponentScan Annotation**